

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

02 - Git e GitLab

Prof. Alex Kutzke

Agosto de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Explicar que problema o controle de versão resolve;
2. Descrever as três áreas de um repositório local e dizer em qual delas um arquivo está, a partir da saída de `git status`;
3. Executar o ciclo `add`, `commit` e `push`, e ler o que cada comando devolveu;
4. Escrever mensagens de *commit* que descrevam a alteração feita;
5. Ler o histórico com `git log`, `git show` e `git diff`;
6. Resolver um conflito de mesclagem editando o arquivo marcado pelo Git;
7. Entregar um exercício da disciplina pelo GitLab, do *fork* ao *push*.

Dinâmica da aula

Aula de mãos no teclado. Abra agora um terminal e deixe-o visível ao lado dos slides.

No seu terminal

Os quadros verdes são o que você digita. Digite junto com o professor e espere a explicação antes de seguir para o próximo.

Entre um comando e outro, paramos para olhar a saída. Ler o que o Git respondeu é metade do aprendizado desta aula: quase todo problema com Git na disciplina vem de alguém que digitou sem ler a resposta.

Roteiro

Preparação

Ciclo local

Histórico

Remoto

Conflito

Exercícios

O problema

A pasta do trabalho da disciplina anterior:

```
trabalho.odt
trabalho_v2.odt
trabalho_v2_revisado.odt
trabalho_FINAL.odt
trabalho_FINAL_agora_vai.odt
```

O arranjo funciona no sentido mais fraco possível: nada foi perdido.
Fora isso, não responde a nada.

- Qual é a versão mais recente, a FINAL ou a FINAL_agora_vai?
- O que mudou de v2 para v2_revisado?
- Em qual arquivo ainda existe o parágrafo apagado por engano na terça?
- Se o trabalho fosse em dupla, quem escreveu o quê?

Sistema de controle de versão

Guarda o histórico de um conjunto de arquivos. O que ele registra não é o conteúdo atual: é a **sequência de estados** pela qual o conjunto passou, com autoria, data e descrição em cada passo.

Git

Criado em 2005 por Linus Torvalds para o desenvolvimento do núcleo do Linux. Software livre, sistema dominante hoje, e roda inteiramente na sua máquina.

O que o Git resolve nesta disciplina

Histórico. Você quebra o CSS às 22h e não faz ideia do que mexeu. Compare o estado atual com o do último *commit* e veja a linha alterada.

Duas máquinas. Você começa o exercício no laboratório e termina em casa. Sem repositório central, a sincronização vira *pen drive* e anexo de e-mail, e uma hora as duas cópias divergem.

Entrega auditável. O histórico mostra o que você entregou, quando e **como** chegou lá. É por isso que o diário de bordo do trabalho prático é avaliado.

git --version

No seu terminal

```
git --version
```

```
git version 2.55.0
```

Se o comando não for encontrado, instale a partir de

<https://git-scm.com/downloads>

Windows

O instalador oficial traz o **Git Bash**, um terminal onde todos os comandos desta aula funcionam como escritos. Use-o. No PowerShell, alguns exemplos se comportam de outro jeito.

git config

No seu terminal

```
git config --global user.name "Seu Nome Completo"  
git config --global user.email "seuemail@example.com"  
git config --global init.defaultBranch main
```

O autor de cada *commit* vem daí, e não do serviço onde o repositório está hospedado. Sem configuração, o Git inventa algo a partir do usuário do sistema, e o histórico fica assinado por `aluno@lab-a15-pc07`.

Nos computadores do laboratório

A máquina é de todo mundo. Execute os dois primeiros comandos **sem** `--global`, dentro de cada repositório clonado.

Onde cada valor de configuração foi parar

No seu terminal

```
git config --list --show-origin | grep -E "user|defaultBranch"
```

A saída mostra, para cada valor, o arquivo de onde ele veio. Três níveis possíveis, do mais geral ao mais específico:

- do sistema, para todos os usuários da máquina;
- do usuário, o que `--global` escreve;
- do repositório, dentro do próprio `.git/config`.

O mais específico vence. É por isso que a configuração local do laboratório resolve.

git init

No seu terminal

```
mkdir catalogo  
cd catalogo  
git init
```

Initialized empty Git repository in /home/ana/catalogo/.git/

Nenhuma pasta de destino, nenhum servidor, nenhuma conta. Tudo o que o Git fez aconteceu dentro de catalogo/.

O repositório é a pasta .git

No seu terminal

```
ls -a
```

```
.  ..  .git
```

Ali dentro mora o histórico inteiro: todas as versões de todos os arquivos, todos os autores, todas as datas.

- Copiar a pasta catalogo/ copia o repositório inteiro. Apagar .git devolve uma pasta comum, sem histórico nenhum.
- **O Git não precisa de rede.** Tudo até a seção do remoto funciona com o cabo desconectado.

Repositório dentro de repositório

Não execute `git init` dentro de um repositório

Os dois passam a disputar os mesmos arquivos, e a confusão é difícil de desfazer.

Na dúvida sobre onde você está, `git status` antes de `git init`. Se ele responder alguma coisa em vez de reclamar que não há repositório, você já está dentro de um.

git status

No seu terminal

```
echo '<h1>Catálogo de Produtos</h1>' > index.html
git status
```

On branch main

No commits yet

Untracked files:

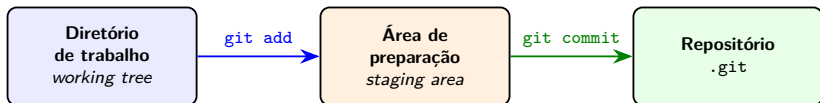
(use "git add <file>..." to include in what will be committed)

index.html

nothing added to commit but untracked files present

Não rastreado (*untracked*): o Git nunca viu esse arquivo e não vai gravá-lo sem ordem explícita.

As três áreas



- **Diretório de trabalho:** a pasta como o editor de texto a vê.
- **Área de preparação:** a lista do que vai entrar no próximo *commit*.
- **Repositório:** o histórico já gravado.

Para que serve a área de preparação

É a parte que causa mais estranheza no primeiro contato, porque parece um passo burocrático a mais.

Ela existe para você escolher **o que entra em cada commit**.

Você mexeu em cinco arquivos: dois resolvem um problema e três resolvem outro. Dá para gravar dois *commits* separados, cada um com sua mensagem, em vez de um pacote único e ininteligível.

git add e git commit

No seu terminal

```
git add index.html
git status
git commit -m "Cria página inicial do catálogo"
```

Entre um comando e outro, leia: o index.html saiu de Untracked files e apareceu sob Changes to be committed.

```
[main (root-commit) 5d09b83] Cria página inicial do catálogo
1 file changed, 1 insertion(+)
create mode 100644 index.html
```

O que um commit guarda

Um *commit* é o registro de um **estado completo** do projeto, e não a lista do que mudou. O Git guarda a fotografia de todos os arquivos naquele instante.

- o conteúdo de todos os arquivos versionados;
- o **autor**, com nome e e-mail;
- a **data**;
- a **mensagem** que você escreveu;
- o **commit anterior**, chamado de *pai*;
- um **identificador** de 40 caracteres hexadecimais, o *hash*.

Internamente o Git evita duplicar o conteúdo de arquivos que não mudaram, mas isso é otimização de armazenamento e não altera o modelo.

O identificador é calculado a partir de todo o resto

Conteúdo, autor, data, mensagem e pai entram na conta. Alterar qualquer um produz um identificador diferente.

Daí vêm duas coisas:

- a **integridade** do histórico, já que não dá para trocar o passado sem que os identificadores mudem;
- o trabalho que dá reescrever o histórico de um repositório compartilhado.

O campo *pai* é o que dá ao histórico a forma de corrente: cada *commit* aponta para o anterior, até o primeiro registro.

git log

No seu terminal

```
git log
git log --oneline
```

```
commit 5d09b83191365b932a94e1c90b68b58d5fe12c55
Author: Ana Souza <ana@example.com>
Date:   Sat Aug 8 16:51:53 2026 -0300
```

Cria página inicial do catálogo

Ninguém digita os 40 caracteres. Os sete primeiros bastam para identificar o *commit*, e é o que o `--oneline` mostra.

Mensagens de commit

A mensagem é o único campo que depende de você.

9b1c2d7 update

4a8e330 asdf

c1d0f92 correções

9b1c2d7 Corrige alinhamento dos cards em telas estreitas

4a8e330 Adiciona validação de e-mail no formulário de contato

c1d0f92 Substitui imagens placeholder pelas fotos definitivas

O segundo histórico permite achar quando algo entrou sem abrir um único arquivo.

Convenções para as entregas da disciplina

- **Escreva o que o commit faz**, com verbo no início e no presente do indicativo: “Adiciona”, “Corrige”, “Remove”, “Atualiza”.
- **Uma linha de até cerca de 50 caracteres.** Precisando de mais, o `git commit` sem `-m` abre um editor: título, linha em branco, corpo.
- **Um commit por alteração com sentido próprio.** Nem um *commit* por letra digitada, nem um *commit* único no fim da aula chamado entrega.

Modificado contra não rastreado

No seu terminal

```
echo '<p>Em construção.</p>' » index.html
echo 'body { font-family: sans-serif; }' > estilo.css
git status
```

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes...)
modified: index.html

Untracked files:

estilo.css

O Git já conhece o index.html, e percebeu que ele mudou. O arquivo estilo.css ele nunca viu antes.

Preparar um arquivo só

No seu terminal

```
git add index.html  
git commit -m "Adiciona aviso de página em construção"  
git status --short
```

```
?? estilo.css
```

Arquivo que não passou por `git add` não está no *commit*. Essa é a causa mais comum de “entreguei e o professor disse que faltava arquivo”.

git status --short

Duas colunas: a primeira é a área de preparação, a segunda é o diretório de trabalho. Abaixo, _ representa uma coluna em branco; no terminal você verá um espaço.

Código	Significado
??	Arquivo não rastreado
_M	Modificado, ainda não preparado
M_	Modificado e preparado
A_	Arquivo novo, já preparado
MM	Preparado e modificado de novo depois disso

git status é o comando que você mais vai usar. Execute-o antes de cada add, antes de cada commit e depois de cada push.

git show

No seu terminal

```
git show
```

```
commit 7f95d857f84c74ac5325759dcf0ac9de11ed1816
Author: Ana Souza <ana@example.com>
Date:   Sat Aug 8 16:52:03 2026 -0300
```

Adiciona aviso de página em construção

```
diff --git a/index.html b/index.html
index 434dae7..cede13d 100644
--- a/index.html
+++ b/index.html
@@ -1,2 @@
   <h1>Catálogo de Produtos</h1>
+<p>Em construção.</p>
```

O formato diff

- linhas com + foram acrescentadas;
- linhas com - foram removidas;
- linhas sem marca são contexto, mostradas para você se localizar.

A linha @@ -1 +1,2 @@ é o cabeçalho do pedaço (*hunk*) que vem logo abaixo. Um arquivo com alterações em pontos distantes gera vários pedaços, cada um com seu cabeçalho.

```
@@ -início,quantidade +início,quantidade @@
```

O - se refere à versão antiga do arquivo, o + à nova.

Lendo @@ -1 +1,2 @@

- -1: no arquivo antigo, o pedaço começa na linha 1 e tem 1 linha. Quando a quantidade é 1, o Git omite o número.
- +1,2: no arquivo novo, o mesmo pedaço começa na linha 1 e passou a ter 2 linhas.

Em um arquivo maior você veria algo como @@ -12,7 +12,9 @@: sete linhas a partir da linha 12 viraram nove linhas a partir da linha 12.

Os números localizam a alteração dentro do arquivo. Eles não contam quantas linhas mudaram.

git diff e git diff --staged

No seu terminal

```
echo '<p>Segunda linha.</p>' » index.html
git diff
git add index.html
git diff
git diff --staged
```

O segundo `git diff` não devolve nada. O `--staged` mostra a linha acrescentada.

- `git diff`: o que eu mexi e ainda não marquei.
- `git diff --staged`: o que vai entrar se eu der `commit` agora.

Desfazer antes do commit

Tirar da área de preparação, mantendo as alterações no diretório de trabalho:

```
git restore --staged index.html
```

Descartar as alterações e voltar ao estado do último *commit*:

```
git restore index.html
```

git restore sem --staged apaga o seu trabalho e não tem desfazer

O conteúdo descartado nunca foi gravado em nenhum *commit*, então não existe cópia em lugar nenhum. Confira com `git diff` antes.

Desfazer depois do commit

Envolve comandos fora do escopo desta disciplina, como o `git revert` e o `git reset`.

Saída suficiente para o nosso caso: faça um *commit* novo que corrija o problema.

O histórico registra o erro e a correção, o que é honesto e, num diário de bordo, é exatamente o que se quer ver.

.gitignore

No seu terminal

```
echo 'minha-senha-secreta' > senha.txt
git status --short
printf 'senha.txt\n*.log\n' > .gitignore
git status --short
git check-ignore -v senha.txt
```

```
.gitignore:1:senha.txt  senha.txt
```

O check-ignore responde qual linha de qual arquivo está escondendo o arquivo. Senhas, temporários do editor, saída de compilação e bibliotecas baixadas ficam de fora do repositório.

Duas ressalvas sobre o .gitignore

- O próprio .gitignore deve ser versionado, para valer também para quem clonar o repositório.
- Ele só afeta arquivos **não rastreados**. Se você já deu git add num arquivo, acrescentá-lo ao .gitignore depois não o remove do repositório.

No seu terminal

```
git add .gitignore  
git commit -m "Ignora arquivos de senha e de log"
```

Repositório remoto

Uma cópia do repositório hospedada em outro lugar, que serve de ponto de encontro entre as suas máquinas, e entre você e o professor.

Comando	Direção	O que faz
<code>git clone URL</code>	remoto → local	cópia completa, com histórico
<code>git push</code>	local → remoto	envia o que só existe aqui
<code>git pull</code>	remoto → local	traz o que só existe lá e integra

Os três transferem **commits**, e não arquivos soltos. Só vai para o servidor aquilo que passou por `git commit`.

`origin` é o apelido padrão do remoto principal. Não é palavra reservada, é convenção.

O endereço do remoto é uma URL, e o transporte é HTTP

```
https://gitlab.com/ana/catalogo.git
```

Esquema, host e caminho, a mesma estrutura vista na aula sobre HTTP. O `git push` num remoto `https://` faz requisições HTTP, com método, cabeçalhos e código de status.

Isso explica dois comportamentos que você vai encontrar:

- repositório inexistente ou privado devolve erro de autenticação, e o Git pede usuário e senha;
- rede do laboratório com a porta bloqueada faz o `push` falhar por tempo esgotado, sem nada de errado no seu repositório.

Forjas

Além do Git, um serviço de hospedagem oferece interface web, controle de acesso, relatório de problemas e revisão de código. Esse tipo de serviço é chamado de **forja** (*forge*). GitHub, GitLab e Codeberg são forjas.

Nenhum desses recursos faz parte do Git. O Git roda na sua máquina; a forja é um serviço construído em volta dele. Trocar de forja não altera nenhum dos comandos vistos até aqui.

Nesta disciplina usamos o **GitLab**: o plano gratuito permite repositórios privados sem limite de quantidade.

Conta no GitLab

No navegador

1. https://gitlab.com/users/sign_up, com o **e-mail da UFPR** e o seu **GRR como nome de usuário**, em minúsculas (grr20249999). Confirme o e-mail que o GitLab envia;
2. em **Edit profile**, preencha o **nome completo**.

O GRR no nome de usuário e o e-mail institucional resolvem a identificação: o professor sabe de quem é cada repositório sem perguntar. Nome de usuário do tipo `dark_coder_2007` não diz nada a quem corrige.

Quem já usa o GitLab, mantenha a conta pessoal como está e crie esta outra para a disciplina. Os trabalhos ficam separados dos seus projetos.

Grupo

Um **grupo** reúne vários repositórios sob um mesmo controle de acesso. Você cria o seu uma vez no semestre, e todas as tarefas vão para dentro dele.

No navegador

1. menu **+**, **New group**, **Create group**;
2. **Group name**: ds122-2026-2-turno-grr, em minúsculas, com n ou t no lugar de turno. Exemplo:
ds122-2026-2-t-grr20261234;
3. **Visibility level**: **Private**;
4. **Manage** > **Members** > **Invite members**: procure alexkutzke e atribua o papel **Reporter**.

O papel Reporter deixa o professor ler e comentar, sem poder alterar o seu código.

Fork

Uma cópia de um repositório feita **no servidor**, sob a sua conta ou grupo. O original continua onde estava, e você passa a ter um repositório próprio, com o mesmo conteúdo e o mesmo histórico, sobre o qual tem permissão de escrita.

É esse o mecanismo das entregas: o professor publica o repositório com o enunciado, cada aluno faz o *fork* para dentro do próprio grupo, e resolve ali.

O *fork* acontece no site. O clone acontece na sua máquina. A sequência completa é *fork*, clone, add, commit, push.

Fork da tarefa de hoje

No navegador

1. abra o endereço do repositório da tarefa, publicado na UFPR Virtual;
2. clique em **Fork**;
3. em **Project URL**, escolha **o seu grupo**;
4. deixe a visibilidade em **Private** e clique em **Fork project**;
5. copie o endereço **HTTPS** do seu *fork*.

Trabalhando em dupla, apenas um dos dois faz o *fork*, e adiciona o colega ao repositório como developer.

git clone

No seu terminal

```
git clone https://gitlab.com/SEU-GRUPO/nome-da-tarefa.git
cd nome-da-tarefa
git remote -v
git log --oneline
git branch --show-current
```

```
origin  https://gitlab.com/ana/catalogo.git (fetch)
origin  https://gitlab.com/ana/catalogo.git (push)
```

O origin já aponta para o seu *fork*, o histórico veio junto e o ramo atual é main. Nada disso precisou de `git init` nem de `git remote add`.

Primeiro push

No seu terminal

```
git config user.name "Seu Nome Completo"
git config user.email "seuemail@example.com"
git add README.md
git commit -m "Registra os integrantes da dupla"
git push
```

Abra o seu *fork* no navegador e confirme que a alteração está lá. Enquanto não aparecer na página do GitLab, a tarefa não foi entregue.

Quando o remoto ainda não conhece o seu ramo

Num repositório criado do zero com `git init`, o primeiro `push` precisa dizer para onde vai:

```
git push -u origin main
```

```
To https://gitlab.com/ana/catalogo.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

A opção `-u` grava a associação entre o seu `main` local e o `main` do `origin`. Depois disso, `git push` e `git pull` sem argumentos já sabem o destino.

Quem clonou não precisa disso: o `clone` já configurou a associação.

Quando o push é recusado

Você deu push de casa na quarta e, na sexta, no laboratório, está com uma cópia antiga.

```
! [rejected]          main -> main (fetch first)
error: failed to push some refs to 'https://gitlab.com/...'
hint: Updates were rejected because the remote contains work
hint: that you do not have locally.
```

A recusa protege o histórico: aceitar o envio apagaria do servidor os *commits* que você não tem.

```
git pull
git push
```

divergent branches

Numa instalação nova, o `git pull` pode parar assim:

```
hint: You have divergent branches and need to specify how to  
hint: reconcile them.
```

```
fatal: Need to specify how to reconcile divergent branches.
```

O Git está perguntando como juntar as duas linhas do tempo. Escolha a mesclagem, que é a opção adequada aqui:

```
git config --global pull.rebase false  
git pull
```

Hábito que evita o problema

`git pull` ao **começar** a trabalhar, antes de editar qualquer arquivo. `git push` ao terminar.

Quando o Git para e pergunta

O pull anterior funcionou porque as duas máquinas mexeram em arquivos diferentes.

Quando as duas alteram **a mesma linha do mesmo arquivo**, o Git não tem como decidir, e para.

Vamos provocar o caso de propósito, para você ver a mensagem em condições controladas em vez de na véspera da entrega.

Provocar o conflito

No navegador

No seu *fork*, edite o README.md pelo próprio site (ícone de lápis), altere a primeira linha e confirme a alteração.

No seu terminal

```
git commit -am "Altera o título pela máquina local"
git pull
```

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Altere **a mesma linha** que você alterou no site, com outro texto.

Onde está o problema

No seu terminal

```
git status
```

```
You have unmerged paths.
```

```
(fix conflicts and run "git commit")
```

```
(use "git merge --abort" to abort the merge)
```

```
Unmerged paths:
```

```
(use "git add <file>..." to mark resolution)
```

```
both modified:   README.md
```

O `git merge --abort` desfaz a mesclagem inteira e devolve o repositório ao estado anterior ao `pull`. É a saída de emergência.

Os marcadores dentro do arquivo

Abra o README.md no editor. O Git escreveu as duas versões dentro dele:

```
««««< HEAD
# Catálogo Verde
=====
# Catálogo de Produtos da Ana
»»»»> c34ab3031b9f162880e357de2a5bc27e68d9b6ca
```

- entre ««««< HEAD e =====: a **sua** versão local;
- entre ===== e »»»»>: a versão que veio do servidor, identificada pelo *hash* do *commit*.

Resolver

Editar o arquivo até ele ficar como deve ficar, o que inclui **apagar as três linhas de marcador**. Você pode ficar com uma das versões, com a outra, ou escrever uma terceira.

No seu terminal

```
git add README.md
git commit
git push
```

O `git commit` sem `-m`, aqui, abre o editor com uma mensagem de mesclagem já preenchida. Basta salvar e sair.

Conflito não é erro nem defeito do Git. É o Git recusando-se a escolher por você entre duas alterações incompatíveis. O erro seria resolvê-lo apagando o trabalho do colega sem ler.

Entrega

Estes exercícios valem nota

Compõem o item *Exercícios em sala* da média, e a entrega é até o final do encontro de hoje.

A entrega é o próprio *fork* no GitLab, com os *commits* pedidos e o *push* feito. **Não há link a enviar em lugar nenhum:** o professor recolhe os repositórios pelo nome do grupo.

Três coisas precisam estar certas: o nome do grupo, o papel de reporter do alexkutzke e o *fork* dentro do grupo. Grupo fora do padrão não é encontrado, e o que não é encontrado não é corrigido.

Como trabalhar

Em dupla, no formato de **Mob Programming**: um dos dois faz o *fork* no próprio grupo e adiciona o colega ao repositório como developer. Registrem no README.md o nome completo e o GRR dos dois.

São cinco itens, e todos mexem em pouca coisa: editar o README.md e criar alguns arquivos.

O que está sendo avaliado é o **histórico** que vocês produzirem, e não o conteúdo dos arquivos.

Ao usar IA generativa durante a aula, aplique o pré-prompt sugerido no material da disciplina.

Exercícios 1 a 3

1. *Fork*, clone e configuração de `user.name` e `user.email` dentro do repositório. No `README.md`, nome completo e GRR dos dois integrantes. Grave, envie e confira no navegador.
2. Crie `anotacoes.md` com um resumo, em suas palavras, da diferença entre diretório de trabalho, área de preparação e repositório. *Commit* separado, com mensagem descritiva.
3. Crie `contato.html` e `sobre.html` de uma vez, com uma linha em cada. Prepare **apenas o `contato.html`** e grave. Cole no `README.md` a saída de `git status --short` obtida logo depois, com uma frase explicando por que o `sobre.html` ficou de fora. Grave o segundo *commit* e envie os dois.

Exercícios 4 e 5

4. Crie `rascunho.log` e um `.gitignore` que o esconda. Cole no `README.md` a saída de `git check-ignore -v rascunho.log`. Versione o `.gitignore` e envie.
5. Provoque um conflito: edite o `README.md` pelo site do GitLab, edite a mesma linha na sua máquina, e dê `git pull`. Resolva, grave e envie. Ainda no `README.md`, descreva em até três frases o que o Git mostrou e como vocês decidiram qual versão manter.

Ao terminar, abra o *fork* no navegador e confirme que todos os *commits* estão lá. *Commit* que não foi enviado não existe para quem corrige.

Desafio

Para quem terminar antes.

Um *commit* guarda a fotografia completa do projeto, mas o Git não duplica o conteúdo de arquivos que não mudaram. Descubra o que de fato muda, na estrutura interna, entre dois *commits* seguidos.

```
git cat-file -p HEAD
```

Siga os identificadores que aparecerem na saída, um a um, sempre com `git cat-file -p`.

Resumo

- Controle de versão guarda a sequência de estados de um projeto, com autoria, data e descrição em cada passo.
- O repositório é a pasta `.git`. O Git funciona sem rede; o remoto é opcional.
- Um arquivo está no diretório de trabalho, na área de preparação ou no repositório. O `add` avança uma área, o `commit` avança a outra.
- Arquivo que não passou por `git add` não entra no *commit*.
- O *commit* guarda a fotografia do projeto, o autor, a data, a mensagem, o *commit* pai e um identificador calculado a partir de tudo isso.
- `git status` responde onde você está; `git diff` e `git diff --staged` respondem o que mudou, em relação a áreas diferentes.
- `clone`, `push` e `pull` transferem *commits*, nunca arquivos soltos.
- Conflito acontece quando duas alterações tocam a mesma linha. O Git marca o arquivo e a decisão é sua.

Bibliografia

- CHACON, S.; STRAUB, B. **Pro Git**, 2ª ed., cap. 1, 2 e 5.
<https://git-scm.com/book/pt-br/v2>
- DUDLER, R. **Guia rápido para o Git**.
http://rogerdudler.github.io/git-guide/index.pt_BR.html
- GitLab. **Documentação**.
<https://docs.gitlab.com/>